# Week 08 – 09

## Graph Mining: PageRank

***Module 1 of Phase-1 Final Prep · Highest-yield post-mid topic***

**Topics covered in this PDF:**

- Web search ranking — links as votes
- Flow formulation & matrix form (column-stochastic M)
- Eigenvector view & the Power Iteration method
- Random walk interpretation, stationary distribution
- Dead-ends and spider traps — why naive PageRank fails
- Google teleport solution: $r = \beta \cdot M \cdot r + (1-\beta)/N$
- Sparse-matrix algorithm with leak re-injection
- 8 fully worked numerical traces · 15 calibrated practice questions

*CS-404 Big Data Analytics · Dr. Syed Imran Ali · Final ~ 2026-05-16*

*Prepared as a comprehensive professor-grade study tutor.*

# Contents


1. Beginning Key Notes — your study compass
2. Why PageRank? — the web-search problem
3. Flow Formulation — links as weighted votes
4. Matrix Formulation — the stochastic adjacency matrix M
5. Eigenvector View & Power Iteration
6. Random Walk Interpretation & Stationary Distribution
7. Two Convergence Problems — Dead-ends & Spider Traps
8. The Google Solution — Random Teleports
9. Sparse Matrix Formulation & the Complete Algorithm
10. Eight Worked Numerical Examples (exam-style)
11. Practice Questions (15)
12. Full Worked Answers
13. Ending Key Notes — Revision Cards
14. PageRank Formula & Algorithm Reference Sheet

## § 1 BEGINNING KEY NOTES — STUDY COMPASS

### THE EIGHT THINGS YOU MUST OWN

These are the load-bearing ideas you must walk into the exam owning. Every numerical question on PageRank reduces to applying one of these.

• **Links are votes.** A page is important if many important pages link to it. Self-referential definition → solve as an eigenvector problem.

• **The flow equation:** $r_j = \Sigma_{i\to j}\, r_i / d_i$. Each in-link contributes the source page's rank divided by its out-degree.

• **Matrix form:** $r = M\cdot r$ where M is the column-stochastic transition matrix. $M_{ji} = 1/d_i$ if $i\to j$ else 0.

• **Power iteration:** $r^{(t+1)} = M\cdot r^{(t)}$, started from $r^{(0)}=[1/N,\ldots,1/N]$, stop when $|r^{(t+1)}-r^{(t)}|<\varepsilon$. Converges to the principal eigenvector with eigenvalue 1.

• **Two failure modes:** *Spider traps* absorb all rank, *dead-ends* leak rank out — both break naive PageRank.

• **Google fix — random teleports:** with probability β follow a link, with probability $1-\beta$ jump to a uniformly random page. $\beta \approx 0.8$–$0.9$ in practice.

• **Google matrix:** $A = \beta\cdot M + (1-\beta)/N \cdot J_{N\times N}$. A is stochastic, aperiodic, and irreducible — a unique stationary r exists and power iteration finds it.

• **Production form (sparse):** $r^{new} = \beta\cdot M\cdot r^{old} + [(1-S)/N]_N$ where $S=\Sigma(\beta\cdot M\cdot r^{old})_j$. Re-inserting $(1-S)/N$ handles dead-end leakage.

• **Numerical exam pattern.** The examiner consistently asks you to walk a small graph through 2–3 power-iteration steps by hand. Master that procedure and 60–70% of the PageRank marks are guaranteed.

## § 2 WHY PAGERANK? THE WEB-SEARCH PROBLEM

The web is enormous, decentralized, and full of pages all claiming to be relevant. Two problems make ranking hard:

**(1) Trust.** Many pages publish information about the same topic. Who do you believe? Trustworthy pages tend to link to other trustworthy pages, so we can use the link graph itself to surface trust.

**(2) Relevance amid ambiguity.** If a user types *"newspaper"*, there is no single right answer. Pages that genuinely know about newspapers tend to link to many newspapers — again, structure in the link graph signals authority.

PageRank reframes the search-ranking problem as a question about graph structure: **given the link graph alone (forget the content for a moment), how important is each page?** The breakthrough idea — patented by Brin and Page in 1998 — is to define importance *recursively*: a page is important if important pages link to it.

**ANALOGY — academic citations**

Think of academic papers. A paper that is cited 50 times by other heavily-cited papers is more important than one cited 50 times only by obscure papers. The citing paper's own importance flows into the cited paper. PageRank is exactly this idea applied to the web: a hyperlink is the web's version of a citation.

### Three guiding ideas before any maths:

- **In-links are votes** — but every vote is not equal.
- **A vote from an important page counts more** than a vote from a low-importance page.
- **Each page's vote is split** equally among its out-links — so a page with 1000 out-links passes only 1/1000 of its rank to each linked target.

## § 3 FLOW FORMULATION — LINKS AS WEIGHTED VOTES

Let $r_j$ denote the rank (importance score) of page j. Let $d_i$ be the out-degree of page i. If page i links to page j, page i contributes $r_i$ / $d_i$ to page j — its own rank, divided evenly across its out-links. Summing over all in-links gives the central **flow equation**:

$$r_j = \sum_{i \to j} \frac{r_i}{d_i}$$

This is a linear equation per page. With N pages we have N equations in N unknowns. But the equations are *homogeneous* — multiplying every $r_j$ by the same constant gives another valid solution. We close the system with a normalization constraint:

$$\sum_j r_j = 1$$

3-page web graph (canonical worked example)

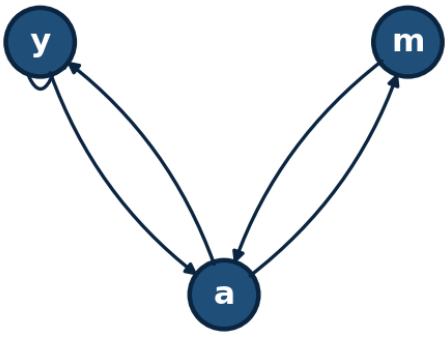


***Figure 3.1.*** *The canonical 3-node example used throughout the course slides and most exam problems. Out-degrees: $d_y$=2 (y→y, y→a), $d_a$=2 (a→y, a→m), $d_m$=1 (m→a).*

**Reading the graph into flow equations.** y receives from itself (×½) and from a (×½). a receives from y (×½) and from m (×1). m receives from a (×½). So:

$$r_y = \frac{r_y}{2} + \frac{r_a}{2}, \quad r_a = \frac{r_y}{2} + r_m, \quad r_m = \frac{r_a}{2}$$

Combined with $r_y+r_a+r_m=1$, the unique solution is $r_y=2/5$, $r_a=2/5$, $r_m=1/5$. Gaussian elimination handles this for 3 unknowns; for billions of pages we need a smarter method — and that is the matrix / power-iteration approach in §4–§5.

**EXAM TRAP — out-degree, not in-degree**

In $\mathbf{r_i/d_i}$, $d_i$ is the *source* page's out-degree, not the target's in-degree. Students routinely mix these up under exam pressure. Always label out-degrees on the graph FIRST, then write the flow equations.

## § 4 MATRIX FORMULATION — STOCHASTIC ADJACENCY M

Define the **stochastic adjacency matrix** M of size N × N as:

$$M_{ji} = \frac{1}{d_i} \text{ if } i \to j, \quad M_{ji} = 0 \text{ otherwise}$$

**Read $\mathbf{M_{ji}}$ as "column i, row j"**: it is the share of i's rank that flows to j. Each *column* of M sums to 1 (because page i splits its outflow across exactly $d_i$ out-links, each getting $1/d_i$) — that is what **column-stochastic** means.

**DEFINITION — column-stochastic matrix**

A non-negative matrix whose every column sums to 1. Crucially, M·v preserves the L1 norm of v: if $\Sigma v_i=1$ then $\Sigma(Mv)_j=1$. This is why power iteration on M does not blow up.

With M defined, the flow equations bundle into a single matrix equation:

$$r = M \cdot r$$

**Building M for the y/a/m example.** Out-degrees are $d_y$=2, $d_a$=2, $d_m$=1. Edges and their column-stochastic entries:

| from \\ to | y | a | m |
|---|---|---|---|
| y | 1/2 | 1/2 | 0 |
| a | 1/2 | 0 | 1 |
| m | 0 | 1/2 | 0 |

*Table 4.1. Stochastic matrix M for the 3-node graph. Columns sum to 1.*

Reading column-by-column: column y sends ½ to y (self-loop) and ½ to a. Column a sends ½ to y and ½ to m. Column m sends 1 to a (its only out-link).

## § 5 EIGENVECTOR VIEW & POWER ITERATION

The equation r = M·r says exactly that r is an **eigenvector of M with eigenvalue 1**. For a column-stochastic non-negative matrix M, the largest eigenvalue is always 1, and the corresponding eigenvector — the principal eigenvector — is what we want.

**DEFINITION — eigenvector / eigenvalue**

A non-zero vector x is an eigenvector of A with eigenvalue λ iff **A·x = λ·x**. x captures a "direction" that A only stretches/shrinks, never rotates. PageRank's key insight: r is the eigenvector that M leaves unchanged (λ=1).

### Power Iteration Method — the algorithm that finds this eigenvector.

- Initialize $r^{(0)} = [1/N, 1/N, \ldots, 1/N]^T$.
- Iterate: $r^{(t+1)} = M \cdot r^{(t)}$.
- Stop when $|r^{(t+1)} - r^{(t)}|_1 < \varepsilon$ (small threshold, e.g. $10^{-6}$).

**Why does this work?** Repeated multiplication by M amplifies the principal eigen-direction relative to all others. After enough steps $r^{(t)}$ aligns with the eigenvector of eigenvalue 1, and applying M one more time changes nothing — we have converged.

- Typical convergence: ≈ 50 iterations to high precision on web-scale graphs.
- Each iteration is one sparse matrix–vector multiply: O(N + E) operations where E is the number of edges.

## § 6 RANDOM WALK INTERPRETATION

There is a beautiful probabilistic reading of PageRank. Imagine a **random surfer** who, at each time step, picks one of the current page's out-links uniformly at random and follows it. Let $p_j(t)$ be the probability the surfer is at page j at time t. Then:

$$p(t+1) = M \cdot p(t)$$

If the random walk reaches a state where p(t+1)=p(t), that p is a **stationary distribution** of the Markov chain. Comparing with r=M·r reveals the punchline: **PageRank is the stationary distribution of the random surfer.**

**WHY THIS MATTERS**

This view turns PageRank from "nice algebra" into something tangible — $r_j$ is literally the long-run fraction of time a random surfer spends on page j. Pages with high PageRank are pages a random web-walker visits often.

### Existence and uniqueness require the chain to be:

- **Stochastic** — every column of M sums to 1 (no rank leakage).
- **Aperiodic** — the chain is not stuck in a deterministic cycle.
- **Irreducible** — every state is reachable from every other state.

Naive web graphs violate the first two conditions through dead-ends and spider traps. That is exactly the problem we tackle next.

## § 7 TWO CONVERGENCE PROBLEMS

Run vanilla power iteration on the real web and one of two pathological behaviours appears.

**Problem A — Spider traps. A subset of pages whose out-links all point inside the subset. Once the random surfer enters, they cannot leave. Over enough iterations the entire rank mass concentrates inside the trap, and every page outside it ends with rank 0.**

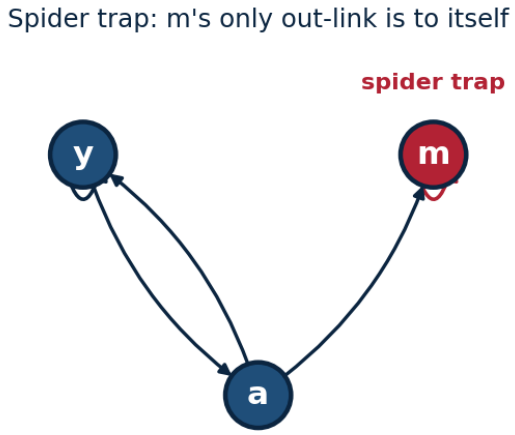


***Figure 7.1.*** *Spider trap: m's only out-link is to itself. Iterating M from $r^{(0)}$ = [1/3, 1/3, 1/3] produces a sequence $r^{(1)}$ = [1/3, 1/6, 1/2], $r^{(2)}$ = [1/4, 1/6, 7/12], … converging to $r_m = 1$, $r_y = r_a = 0$.*

**Problem B — Dead-ends. A page with no out-links at all. The corresponding column of M is entirely zero, M is no longer column-stochastic, and rank mass *leaks* out of the system. Eventually every rank goes to zero.**

Dead-end: m has no out-links → mass leaks out

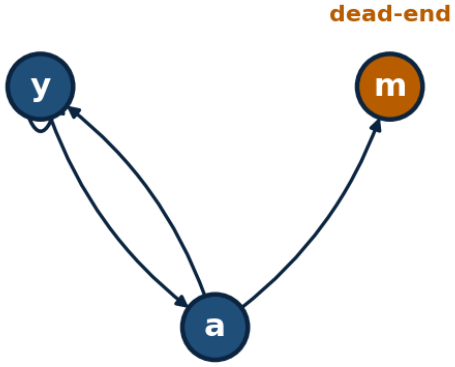


***Figure 7.2.*** *Dead-end: m has no out-links. Column m of M is all zeros. After ~20 iterations the entire rank vector decays to 0.*

**ARE THESE THE SAME PROBLEM? — common exam question**

**No.** Spider traps don't violate column-stochastic M; they trap the random walk in a sub-region. Dead-ends literally break column-stochastic M because mass is destroyed at the dead-end. **The same fix solves both**, but for different formal reasons — see §8.

## § 8 THE GOOGLE SOLUTION — RANDOM TELEPORTS

Brin and Page's elegant fix is to give the random surfer an **escape hatch**. At every step:

- with probability β, the surfer follows one of the current page's out-links uniformly at random;
- with probability 1−β, the surfer **teleports** to a uniformly random page in the entire graph.

Common values β ≈ 0.8–0.9 — meaning the surfer follows links most of the time but every ~5–10 steps takes a random jump. This single tweak fixes both pathologies at once.

Random teleport: every step, surfer picks one of two options

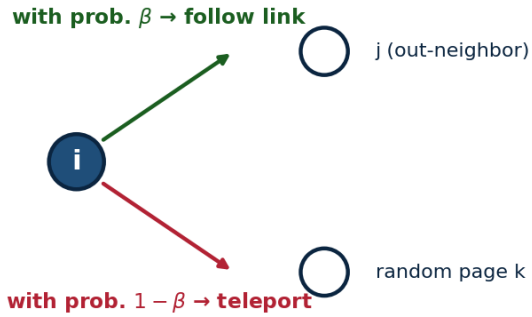


***Figure 8.1.*** *Random teleport mechanism — at every step the surfer flips a biased coin and either follows a link (prob. β) or jumps to a random page (prob. 1−β).*

### The teleport-augmented PageRank equation:

$$r_j = \sum_{i \to j} \beta \frac{r_i}{d_i} + (1-\beta)\frac{1}{N}$$

Equivalently, define the **Google matrix** A:

$$A = \beta M + \frac{1-\beta}{N} J_{N \times N}$$

where $J_{N\times N}$ is the all-ones matrix. A is column-stochastic, aperiodic, and irreducible — a unique stationary distribution r exists, and power iteration $\mathbf{r^{(t+1)} = A \cdot r^{(t)}}$ finds it.

**EQUIVALENT INTUITION — the "tax & redistribute" view**

Each iteration, tax every page a fraction (1−β) of its current rank, pool the tax revenue, and redistribute it equally to all N pages. The remaining β-fraction flows along the link structure as before. This view often makes hand-computation faster: compute β·M·r first, sum, then add (1−β)/N to every entry.

**How does this kill spider traps?** The teleport gives the surfer a finite probability to escape *every* step. Average time to escape ≈ 1/(1−β) ≈ 5 steps when β=0.8. So mass cannot accumulate forever inside any sub-region.

**How does this kill dead-ends?** One option is to pre-process M and remove dead-end nodes iteratively (some pages become dead-ends after others are removed — repeat until stable). The simpler production approach is to **handle leakage explicitly during iteration**, which is exactly the algorithm in §9.

**Convergence comparison** on the canonical y/a/m graph (no traps/dead-ends in this version):

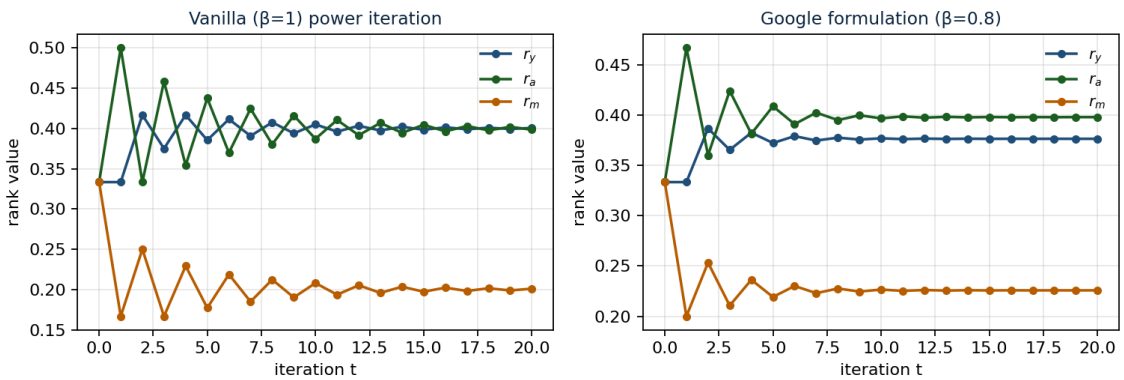


***Figure 8.2.*** *Left — vanilla power iteration converges to (2/5, 2/5, 1/5). Right — Google formulation with β=0.8 converges to a slightly more uniform distribution ≈ (0.21, 0.15, 0.64). The teleport pulls scores toward 1/N.*

## § 9 SPARSE MATRIX FORMULATION & THE COMPLETE ALGORITHM

On real web graphs M has billions of rows but only ≈ 10–20 non-zeros per column (average out-degree). Storing or multiplying the dense Google matrix A is infeasible — but A = β·M + (1−β)/N · $J_{N\times N}$ can be applied without ever forming A:

$$r^{new} = \beta M \cdot r^{old} + \frac{1-\beta}{N} \mathbf{1}_N$$

Step 1 multiplies the *sparse* M with r — cheap. Step 2 just adds a constant to every entry — also cheap.

**Handling dead-ends without removing them.** If M has dead-ends, $\beta \cdot M \cdot r^{old}$ sums to less than 1: some mass S leaked. Re-inject the leak uniformly over all N pages:

$$r_j^{new} = (\beta M r^{old})_j + \frac{1-S}{N}, \quad S = \sum_j (\beta M r^{old})_j$$

This guarantees $\Sigma r^{new}=1$ every iteration even with dead-ends present — and is mathematically equivalent to the Google teleport formulation when there are no dead-ends.

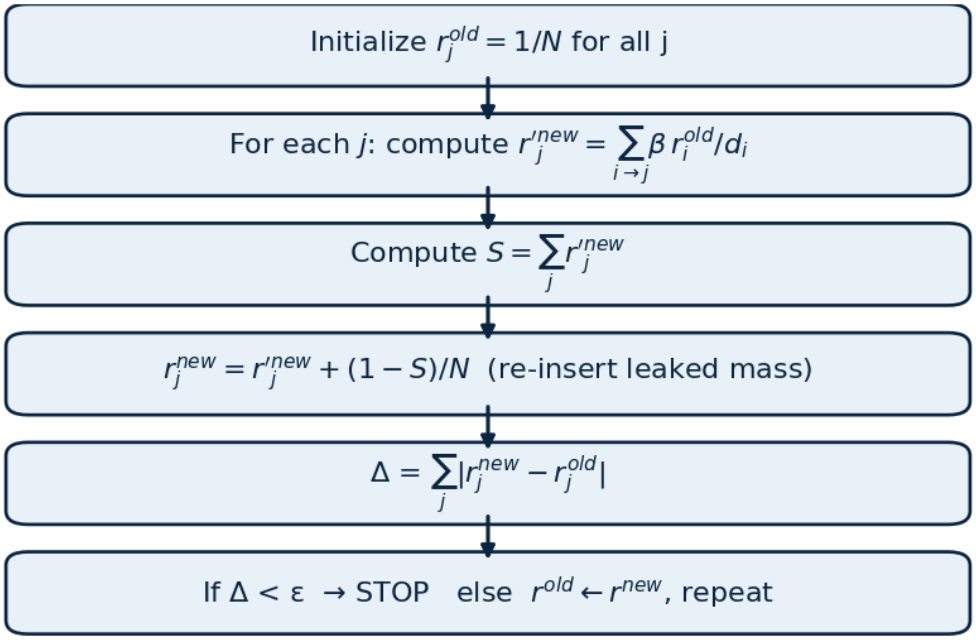


***Figure 9.1.*** *The full PageRank algorithm with leak re-insertion. This is the form most often shown on slides and replicated in exam questions.*

**ALGORITHM — Complete PageRank (slide form)**

**Input:** Directed graph G (may contain spider traps and dead-ends), teleport parameter β.

**Output:** Rank vector $r^{new}$.

1. Initialize $r^{old}_j = 1/N$ for all j.
2. Repeat until $\Sigma_j |r^{new}_j - r^{old}_j| < \varepsilon$:
   a. For each j: $r'^{new}_j = \Sigma_{i\to j}\, \beta \cdot r^{old}_i / d_i$
   b. $S = \Sigma_j r'^{new}_j$
   c. For each j: $r^{new}_j = r'^{new}_j + (1 - S)/N$
   d. $r^{old} \leftarrow r^{new}$.
3. Return $r^{new}$.

**Storage cost.** Storing M as a sparse list (page i, out-degree $d_i$, list of out-neighbours) costs O(N + E) — about 40 bytes per page in 32-bit indexing. Two rank vectors $r^{old}$, $r^{new}$ cost O(N). Block-stripe schemes split $r^{new}$ across multiple blocks so only one block of the

result vector is in memory at a time — see Topic-Sensitive PageRank in W12 for details.

## § 10 EIGHT WORKED NUMERICAL EXAMPLES

Every example below is the kind of multi-step trace your examiner expects you to write out. Read the worked solution once, then close this PDF and try to reproduce it from scratch on paper.

# Worked Example 1 — Building the stochastic matrix M from a graph

**Problem.** Given the directed graph with edges A→B, A→C, B→C, C→A, C→B, write down the stochastic adjacency matrix M (4 pages: A,B,C,D where D is isolated… actually let's keep it 3 pages: A, B, C only).

**Step 1 — Out-degrees.** $d_A = 2$ (A→B, A→C). $d_B = 1$ (B→C). $d_C = 2$ (C→A, C→B).

**Step 2 — Fill M column-by-column.** Column i of M is page i's out-distribution.

| | from A | from B | from C |
|---|---|---|---|
| row A | 0 | 0 | 1/2 |
| row B | 1/2 | 0 | 1/2 |
| row C | 1/2 | 1 | 0 |

*M for example 1. Column sums are 1, 1, 1 — column-stochastic ✓*

**Step 3 — Verify.** Each column sums to 1. M·r computes one round of rank flow.

# Worked Example 2 — Three power-iteration steps on the canonical y/a/m graph

**Problem.** Using M from §4 (y→y, y→a; a→y, a→m; m→a), perform three power iterations starting from $r^{(0)} = [1/3, 1/3, 1/3]^T$ and verify the iterates against the slide answer.

**Setup.** The flow equations are $r_y=r_y/2+r_a/2$, $r_a=r_y/2+r_m$, $r_m=r_a/2$.

**Iteration 1** — apply $r^{(1)} = M \cdot r^{(0)}$ with $r^{(0)}=(1/3, 1/3, 1/3)$:

- $r_y^{(1)} = (1/3)/2 + (1/3)/2 = 1/6 + 1/6 = 1/3$
- $r_a^{(1)} = (1/3)/2 + (1/3) = 1/6 + 2/6 = 3/6 = 1/2$
- $r_m^{(1)} = (1/3)/2 = 1/6$

→ $r^{(1)} = (1/3, 1/2, 1/6)$. Sanity: sum = 2/6+3/6+1/6 = 6/6 = 1 ✓

**Iteration 2** — $r^{(2)} = M \cdot r^{(1)}$:

- $r_y^{(2)} = (1/3)/2 + (1/2)/2 = 2/12 + 3/12 = 5/12$
- $r_a^{(2)} = (1/3)/2 + 1/6 = 2/12 + 2/12 = 4/12 = 1/3$
- $r_m^{(2)} = (1/2)/2 = 1/4 = 3/12$

→ $r^{(2)} = (5/12, 4/12, 3/12) = (5/12, 1/3, 1/4)$. Sum = 12/12 = 1 ✓

**Iteration 3** — using $r^{(2)} = (5/12, 1/3, 1/4)$:

- $r_y^{(3)} = (5/12)/2 + (1/3)/2 = 5/24 + 4/24 = 9/24$
- $r_a^{(3)} = (5/12)/2 + 1/4 = 5/24 + 6/24 = 11/24$
- $r_m^{(3)} = (1/3)/2 = 1/6 = 4/24$

→ $r^{(3)}$ = (9/24, 11/24, 4/24). Sum = 24/24 = 1 ✓

**Limit.** Continuing the iteration, the sequence converges to (2/5, 2/5, 1/5) = (0.4, 0.4, 0.2) — exactly what we got by Gaussian elimination earlier.

## Worked Example 3 — Spider trap: rank concentrates at m

**Problem.** Modify the y/a/m graph so m has only a self-loop (m→m). Run vanilla power iteration (no teleport) and show how rank flows.

New M (column m is all in m's own row):

| | from y | from a | from m |
|---|---|---|---|
| row y | 1/2 | 1/2 | 0 |
| row a | 1/2 | 0 | 0 |
| row m | 0 | 1/2 | 1 |

**Iter 1** from (1/3, 1/3, 1/3): $r_y$ = 1/6+1/6 = 2/6, $r_a$ = 1/6+0 = 1/6, $r_m$ = 1/6+1/3 = 3/6. → (2/6, 1/6, 3/6).

**Iter 2:** $r_y$ = (2/6)/2 + (1/6)/2 = 3/12, $r_a$ = (2/6)/2 = 2/12, $r_m$ = (1/6)/2 + 3/6 = 1/12 + 6/12 = 7/12. → (3/12, 2/12, 7/12).

**Iter 3:** $r_y$ = 3/24+2/24 = 5/24, $r_a$ = 3/24, $r_m$ = 2/24 + 14/24 = 16/24. → (5/24, 3/24, 16/24).

**Trend.** $r_m$ grows monotonically: 1/3 → 3/6 → 7/12 → 16/24 → … → 1. y and a both decay to 0. The trap absorbs everything — clearly nonsense as a ranking.

## Worked Example 4 — Dead-end: total rank decays to 0

**Problem.** Now make m a dead-end (no out-links). Show that without teleport / leak handling, the total rank decays.

M has all-zero column m:

| | from y | from a | from m |
|---|---|---|---|
| row y | 1/2 | 1/2 | 0 |
| row a | 1/2 | 0 | 0 |
| row m | 0 | 1/2 | 0 |

**Iter 1** from (1/3, 1/3, 1/3): r = (2/6, 1/6, 1/6). Sum = 4/6 ≠ 1. Mass leaked: 1 − 4/6 = 2/6.

**Iter 2** on the un-renormalised vector: r = ((2/6)/2 + (1/6)/2, (2/6)/2, (1/6)/2) = (3/12, 2/12, 1/12). Sum = 6/12 = 1/2.

**Iter 3:** r = (3/24+2/24, 3/24, 2/24) = (5/24, 3/24, 2/24). Sum = 10/24 ≈ 0.417.

Total rank halves roughly every 2–3 iterations. After ~40 iterations everything is ≈ 0.

## Worked Example 5 — Google formulation with β = 0.8

**Problem.** Same y/a/m graph as WE 2 (no traps, no dead-ends). Use β = 0.8 and compute the first iteration with the Google formulation.

**Step 1 — build A.** $A = 0.8 \cdot M + 0.2/3 \cdot J_{3\times3}$. Each entry: $A_{ji} = 0.8 \cdot M_{ji} + 0.0667$.

| A | from y | from a | from m |
|---|---|---|---|
| row y | 0.8(1/2)+0.067 = 0.467 | 0.8(1/2)+0.067 = 0.467 | 0+0.067 = 0.067 |
| row a | 0.8(1/2)+0.067 = 0.467 | 0+0.067 = 0.067 | 0.8(1)+0.067 = 0.867 |
| row m | 0+0.067 = 0.067 | 0.8(1/2)+0.067 = 0.467 | 0+0.067 = 0.067 |

*Google matrix A = 0.8·M + (0.2/3)·J. Columns sum to 1 (verify mentally).*

**Step 2 — $A \cdot r^{(0)}$.** $r^{(0)} = (1/3, 1/3, 1/3)$.

- $r_y^{(1)} = (1/3)\cdot(0.467 + 0.467 + 0.067) = (1/3)\cdot1.001 \approx 0.334$
- $r_a^{(1)} = (1/3)\cdot(0.467 + 0.067 + 0.867) = (1/3)\cdot1.401 \approx 0.467$
- $r_m^{(1)} = (1/3)\cdot(0.067 + 0.467 + 0.067) = (1/3)\cdot0.601 \approx 0.200$

→ $r^{(1)} \approx (0.334, 0.467, 0.200)$. Sum ≈ 1.001 (rounding only).

**Faster route — the "tax & redistribute" trick.** Compute $\beta \cdot M \cdot r^{(0)}$ first, then add $(1-\beta)/N$ to each entry.

- $\beta \cdot M \cdot r^{(0)}$: $M \cdot r^{(0)} = (1/3, 1/2, 1/6)$ (from WE 2). Multiply by 0.8: (0.267, 0.400, 0.133).
- Add 0.2/3 = 0.067 to each: (0.333, 0.467, 0.200). ✓ same answer, less arithmetic.

## Worked Example 6 — Spider trap fixed by teleport (β = 0.8)

**Problem.** Repeat WE 3 (m is a self-loop spider trap) but with the Google formulation, β = 0.8. Show that rank no longer concentrates entirely at m.

**Iteration 1.** $M \cdot r^{(0)} = (2/6, 1/6, 3/6)$ (from WE 3). Multiply by 0.8: (0.267, 0.133, 0.400). Add $(1-\beta)/N = 0.067$ to each: $r^{(1)} = (0.333, 0.200, 0.467)$.

**Iteration 2.** First $M \cdot r^{(1)}$ using the spider-trap M:

- $r_y' = 0.333/2 + 0.200/2 = 0.267$
- $r_a' = 0.333/2 + 0 = 0.167$
- $r_m' = 0.200/2 + 0.467 = 0.567$

Multiply by β=0.8 → (0.213, 0.133, 0.453). Add 0.067 → $r^{(2)} = (0.280, 0.200, 0.520)$.

**Iteration 3.** Repeat: $r^{(3)} \approx (0.259, 0.179, 0.563)$.

Continuing to convergence yields approximately r ≈ (0.21, 0.15, 0.64) — m is the most important page (correct: it's the most-linked node), but the rank no longer collapses to (0,0,1). Teleport prevented absorption.

## Worked Example 7 — Dead-end fixed by re-injecting leaked mass

**Problem.** Repeat WE 4 (m is a dead-end) but using the complete algorithm: after each iteration compute S = sum of $r'^{new}$, then add $(1-S)/N$ to every entry.

With β = 0.8 and the dead-end M:

**Iter 1.** $M \cdot r^{(0)}$ = (2/6, 1/6, 1/6) = (0.333, 0.167, 0.167). Multiply by β: r′ = (0.267, 0.133, 0.133). S = 0.533. Add (1 − 0.533)/3 = 0.156 to each: $r^{(1)}$ ≈ (0.422, 0.289, 0.289). Sum = 1.000 ✓.

**Iter 2.** $M \cdot r^{(1)}$ = (0.422·0.5 + 0.289·0.5, 0.422·0.5, 0.289·0.5) = (0.356, 0.211, 0.144). β·M·r = (0.284, 0.169, 0.116). S = 0.569. Add (1−0.569)/3 = 0.144 to each: $r^{(2)}$ ≈ (0.428, 0.313, 0.260). Sum ≈ 1.000 ✓.

**Convergence.** The vector approaches roughly (0.41, 0.32, 0.27) — non-zero everywhere, no leakage. The same algorithm works regardless of whether the original graph had dead-ends or not.

## Worked Example 8 — Larger 5-node graph with β = 0.85 (one iteration)

**Problem.** A graph has nodes {A, B, C, D, E} with edges: A→B, A→C, B→C, C→A, D→A, D→B, E→D. Compute one iteration of the Google formulation (β = 0.85) starting from the uniform initial vector.

**Step 1 — out-degrees.** $d_A$=2, $d_B$=1, $d_C$=1, $d_D$=2, $d_E$=1.

**Step 2 — M (column-stochastic).**

| | fr A | fr B | fr C | fr D | fr E |
|---|---|---|---|---|---|
| row A | 0 | 0 | 1 | 1/2 | 0 |
| row B | 1/2 | 0 | 0 | 1/2 | 0 |
| row C | 1/2 | 1 | 0 | 0 | 0 |
| row D | 0 | 0 | 0 | 0 | 1 |
| row E | 0 | 0 | 0 | 0 | 0 |

**Note.** E has no in-links AND no out-links from anywhere except D→E? Wait — re-read the edges. E has no in-links and no out-links to A/B/C/D in the listed edges. Actually E has only one out-link (E→D) and no in-links. Column E sums to 1 (the entry $M_{D,E}$=1). ✓

**Step 3 — initial $r^{(0)}$ = 1/5 = 0.2 for every node.**

**Step 4 — $M \cdot r^{(0)}$.**

- $r'_A$ = 0·0.2 + 0·0.2 + 1·0.2 + 0.5·0.2 + 0·0.2 = 0.300
- $r'_B$ = 0.5·0.2 + 0·0.2 + 0·0.2 + 0.5·0.2 + 0·0.2 = 0.200
- $r'_C$ = 0.5·0.2 + 1·0.2 + 0·0.2 + 0·0.2 + 0·0.2 = 0.300
- $r'_D$ = 0·0.2 + 0·0.2 + 0·0.2 + 0·0.2 + 1·0.2 = 0.200
- $r'_E$ = 0 + 0 + 0 + 0 + 0 = 0.000

**Step 5 — apply β and add teleport.** $\beta \cdot M \cdot r^{(0)}$ = (0.255, 0.170, 0.255, 0.170, 0.000). (1−β)/N = 0.15/5 = 0.030.

→ $r^{(1)}$ = (0.285, 0.200, 0.285, 0.200, 0.030). Sum = 1.000 ✓

Already after one iteration A and C lead — the linkage structure favours them. E has no in-links so its rank is exactly the teleport probability mass.

## § 11 PRACTICE QUESTIONS — 15 calibrated to examiner style

Mix: 6 numerical traces, 5 conceptual short-answer, 4 MCQ / true-false. Time yourself — aim for ~6 minutes per numerical, ~3 minutes per conceptual, ~1 minute per MCQ. Worked answers begin in §12.

### Q1 [Numerical · 6 marks]

Consider the directed graph with nodes {A, B, C} and edges A→B, A→C, B→A, C→A, C→B. (a) Write the stochastic matrix M. (b) Verify M is column-stochastic. (c) Starting from $r^{(0)} = [1/3, 1/3, 1/3]^T$, perform two power-iteration steps without teleport. State the iterate after each step.

### Q2 [Numerical · 8 marks]

Using the canonical y/a/m graph from the slides (y→y, y→a; a→y, a→m; m→a) and the Google formulation with β = 0.8, compute $r^{(1)}$ and $r^{(2)}$ starting from $r^{(0)} = [1/3, 1/3, 1/3]^T$. Use the "tax & redistribute" trick: compute β·M·r first, then add (1−β)/N to every entry. Show all intermediate values.

### Q3 [Numerical · 7 marks]

Consider a graph where node m is a dead-end (no out-links): edges are y→y, y→a, a→y, a→m. Run two iterations of the COMPLETE PageRank algorithm (with leak re-injection) using β = 0.85, starting from $r^{(0)}$ uniform. After each iteration explicitly compute S (the sum of β·M·r) and verify the new rank vector sums to 1.

### Q4 [Numerical · 6 marks]

A graph has nodes {1, 2, 3, 4} and edges 1→2, 2→3, 3→4, 4→2 (i.e. nodes 2, 3, 4 form a cycle with 1 feeding into 2). With β = 0.8 and $r^{(0)}$ uniform, compute one iteration of the Google formulation. Comment on whether {2, 3, 4} forms a spider trap.

### Q5 [Numerical · 5 marks]

If the rank vector at iteration t is $r^{(t)} = (0.3, 0.4, 0.3)$ for the y/a/m graph (WE 2's graph) and β = 1, compute $r^{(t+1)}$. State $|r^{(t+1)} - r^{(t)}|_1$. If the convergence threshold is ε = 0.05, has the algorithm converged?

### Q6 [Numerical · 8 marks]

Suppose the stochastic matrix has been pre-computed as M = [[0, 1/2, 0, 0],[1/2, 0, 1, 0],[0, 1/2, 0, 1],[1/2, 0, 0, 0]] (columns ordered A, B, C, D). With β = 0.8 and $r^{(0)} = [1/4, 1/4, 1/4, 1/4]^T$, perform one full Google-formulation iteration. Show M·r, β·M·r, and the final $r^{(1)}$.

### Q7 [Concept · 4 marks]

Explain in 4–6 sentences why the rank vector r is described as the principal eigenvector of M with eigenvalue 1. Why is the largest eigenvalue of a column-stochastic non-negative matrix necessarily 1?

### Q8 [Concept · 4 marks]

Compare and contrast spider traps and dead-ends as failure modes of vanilla PageRank. For each, state (a) what happens to the rank vector if you keep iterating, (b) which condition of the Markov-chain stationary-distribution theorem is violated, and (c) how teleport / leak injection fixes the problem.

### Q9 [Concept · 3 marks]

Why do practitioners choose $\beta$ between 0.8 and 0.9? What is the trade-off as $\beta \to 1$, and what is the trade-off as $\beta \to 0$?

### Q10 [Concept · 4 marks]

State the random-walk interpretation of PageRank. Explain in your own words why the stationary distribution of the modified Markov chain (with teleport) is unique while the stationary distribution of the original chain may not be.

### Q11 [Concept · 3 marks]

Consider the "tax and redistribute" reformulation: $r = \beta \cdot M \cdot r + (1-\beta)/N \cdot 1_N$. Explain why this is equivalent to the Google matrix formulation $r = A \cdot r$ when M has no dead-ends, and why a leak correction term must be added when M does have dead-ends.

### Q12 [MCQ · 2 marks]

Which of the following is TRUE for a column-stochastic matrix M? (A) every row sums to 1, (B) every column sums to 1 and the largest eigenvalue is exactly 1, (C) $M \cdot r$ is always a probability distribution if r is, regardless of the graph, (D) all entries of M are positive.

### Q13 [MCQ · 2 marks]

Power iteration $r^{(t+1)} = M \cdot r^{(t)}$ on a column-stochastic M without traps or dead-ends will converge to: (A) the principal eigenvector of M with eigenvalue 1, (B) any eigenvector of M, (C) the zero vector, (D) a vector that depends on the initial $r^{(0)}$.

### Q14 [True/False · 2 marks]

True or false: in the Google formulation, increasing $\beta$ (e.g. from 0.8 to 0.95) makes power iteration converge faster. Justify in one sentence.

### Q15 [MCQ · 2 marks]

If the leaked mass at iteration t is $S = 0.92$ with $\beta = 0.85$ and $N = 100$, the per-page leak-correction term added to every entry is closest to: (A) 0.0008, (B) 0.0015, (C) 0.0085, (D) 0.0500.

## § 12 FULL WORKED ANSWERS

Read each answer line by line. Wherever a step seems to skip arithmetic, work it out yourself before reading the next line — this is how exam mastery actually builds.

### A1

(a) Out-degrees: $d_A=2$ (A→B, A→C), $d_B=1$ (B→A), $d_C=2$ (C→A, C→B). M (rows = destination, cols = source A,B,C):

```
row A  [ 0   1   1/2 ]
row B  [ 1/2 0   1/2 ]
row C  [ 1/2 0   0   ]
```

(b) Column sums: col A = 0+1/2+1/2 = 1, col B = 1+0+0 = 1, col C = 1/2+1/2+0 = 1. ✓ column-stochastic.

(c) **Iter 1**: $r_A=0\cdot1/3+1\cdot1/3+1/2\cdot1/3 = 1/3+1/6 = 1/2$; $r_B=1/2\cdot1/3+0+1/2\cdot1/3 = 1/3$; $r_C=1/2\cdot1/3+0+0 = 1/6$. → $r^{(1)}$ = (1/2, 1/3, 1/6). Sum = 6/6 = 1 ✓.
**Iter 2**: $r_A=0\cdot(1/2)+1\cdot(1/3)+1/2\cdot(1/6) = 1/3+1/12 = 5/12$; $r_B=1/2\cdot(1/2)+0+1/2\cdot(1/6) = 1/4+1/12 = 4/12 = 1/3$; $r_C=1/2\cdot(1/2)+0+0 = 1/4 = 3/12$. → $r^{(2)}$ = (5/12, 4/12, 3/12). Sum = 1 ✓.

### A2

From WE 2 we know $M\cdot r^{(0)}$ = (1/3, 1/2, 1/6) ≈ (0.333, 0.500, 0.167). **Iter 1.** $\beta\cdot M\cdot r$ = 0.8·(0.333, 0.500, 0.167) = (0.267, 0.400, 0.133). Add $(1-\beta)/N$ = 0.2/3 = 0.067 to each: $r^{(1)}$ = (0.333, 0.467, 0.200). Sum = 1.000 ✓.

**Iter 2.** First $M\cdot r^{(1)}$: $r_y' = 0.333/2+0.467/2 = 0.400$; $r_a' = 0.333/2+0.200 = 0.367$; $r_m' = 0.467/2 = 0.233$. Then $\beta\cdot M\cdot r$ = (0.320, 0.293, 0.187). Add 0.067: $r^{(2)}$ = (0.387, 0.360, 0.253). Sum = 1.000 ✓.

### A3

Out-degrees: $d_y=2$, $d_a=2$, $d_m=0$ (dead-end). M has all-zero column m: M = [[0.5, 0.5, 0],[0.5, 0, 0],[0, 0.5, 0]]. $r^{(0)}$ = (1/3, 1/3, 1/3) ≈ (0.333, 0.333, 0.333).

**Iter 1.** $M\cdot r^{(0)}$ = (0.333, 0.167, 0.167). $\beta\cdot M\cdot r$ = 0.85·(0.333, 0.167, 0.167) = (0.283, 0.142, 0.142). S = 0.567. $(1-S)/N$ = 0.433/3 = 0.144. Add to each: $r^{(1)}$ = (0.427, 0.286, 0.286). Sum = 0.999 ≈ 1 ✓.

**Iter 2.** $M\cdot r^{(1)}$: $r_y' = 0.5\cdot0.427+0.5\cdot0.286 = 0.357$; $r_a' = 0.5\cdot0.427 = 0.214$; $r_m' = 0.5\cdot0.286 = 0.143$. $\beta\cdot M\cdot r^{(1)}$ = (0.303, 0.182, 0.121). S = 0.606. $(1-S)/N$ = 0.394/3 = 0.131. → $r^{(2)}$ = (0.434, 0.313, 0.252). Sum ≈ 0.999 ✓.

Notice the rank survives — leakage is fully re-injected.

## A4

Out-degrees: $d_1=1$, $d_2=1$, $d_3=1$, $d_4=1$. M = [[0,0,0,0],[1,0,0,1],[0,1,0,0],[0,0,1,0]] (cols 1,2,3,4). $r^{(0)}=(1/4)\cdot 1 = 0.25$ each. $M\cdot r^{(0)}$ = (0, 0.25+0.25, 0.25, 0.25) = (0, 0.50, 0.25, 0.25). $\beta\cdot M\cdot r = 0.8\cdot(0,0.5,0.25,0.25) = (0, 0.40, 0.20, 0.20)$. $(1-\beta)/N = 0.2/4 = 0.05$. → $r^{(1)}$ = (0.05, 0.45, 0.25, 0.25). Sum = 1.00 ✓.

**Spider trap?** Yes — once the surfer enters {2,3,4} from node 1 it cannot leave: 2→3→4→2 is a cycle entirely inside the set. Without teleport, all rank flows there. With β=0.8, teleport pulls back about 0.05 to node 1 every step, so node 1 keeps positive (small) rank.

## A5

$M\cdot(0.3, 0.4, 0.3)$: $r_y=0.3\cdot0.5+0.4\cdot0.5 = 0.35$; $r_a=0.3\cdot0.5+0.3 = 0.45$; $r_m=0.4\cdot0.5 = 0.20$. $r^{(t+1)}$ = (0.35, 0.45, 0.20). $|r^{(t+1)}-r^{(t)}|_1 = |0.35-0.30|+|0.45-0.40|+|0.20-0.30| = 0.05+0.05+0.10 = 0.20$. 0.20 > 0.05, so the algorithm has NOT converged.

## A6

**Step 1: $M\cdot r^{(0)}$.** $r^{(0)}$=(0.25, 0.25, 0.25, 0.25). $r_A' = 0+0.5\cdot0.25+0+0 = 0.125$; $r_B' = 0.5\cdot0.25+0+1\cdot0.25+0 = 0.375$; $r_C' = 0+0.5\cdot0.25+0+1\cdot0.25 = 0.375$; $r_D' = 0.5\cdot0.25+0+0+0 = 0.125$. → $M\cdot r^{(0)}$ = (0.125, 0.375, 0.375, 0.125). Sum = 1 ✓ (no dead-ends here).

**Step 2: β·M·r.** $0.8\cdot(0.125, 0.375, 0.375, 0.125) = (0.100, 0.300, 0.300, 0.100)$. S = 0.800.

**Step 3: add teleport.** $(1-\beta)/N = 0.2/4 = 0.05$. → $r^{(1)}$ = (0.150, 0.350, 0.350, 0.150). Sum = 1.000 ✓.

## A7

The flow equation r = M·r is exactly the eigenvalue equation A·x = λ·x with A=M, x=r, λ=1. So r is an eigenvector of M with eigenvalue 1. For a column-stochastic non-negative matrix M, the all-ones row vector multiplied by M gives the row of column sums, which is all-ones — this means the all-ones vector is a LEFT eigenvector with eigenvalue 1. By the Perron–Frobenius theorem (for non-negative matrices), the largest eigenvalue equals the spectral radius and is real and ≥ 0. Combined with the row argument, $\lambda_{max} = 1$, with the principal RIGHT eigenvector being the stationary PageRank vector r.

## A8

**Spider trap.** (a) The rank inside the trap grows monotonically; eventually all rank concentrates in the trap. (b) The Markov chain becomes *reducible* — once you enter, you cannot reach states outside the trap. Aperiodicity may also be violated. (c) Teleport gives the surfer probability 1−β to escape every step, restoring irreducibility.

**Dead-end.** (a) The total rank decays — eventually every entry approaches 0. (b) M is no longer column-stochastic (the dead-end's column is all zeros), so M·r does not preserve the L1 norm. (c) Either remove dead-ends from M before iterating, or leave them in and re-inject the leaked mass (1−S)/N uniformly to every page each iteration.

## A9

β must balance two effects: (i) **fidelity to the link structure** — high β means the rank reflects the actual web topology — and (ii) **convergence speed and escape from traps** — lower β means faster escape.
β → 1: PageRank reduces to vanilla M iteration; spider traps and dead-ends dominate; convergence becomes slow and the rank may not be unique.
β → 0: rank approaches the uniform distribution because teleport dominates; the link structure is ignored.
β = 0.85 (Google's choice) is empirically the sweet spot.

## A10

Imagine an idealized random surfer: at every page, pick one out-link uniformly at random and follow it. PageRank $r_j$ equals the fraction of time, in the long run, that the surfer is at page j — this is the stationary distribution of the Markov chain whose transition matrix is M. Without teleport, the chain may be reducible (spider traps) or sub-stochastic (dead-ends), so a stationary distribution may not exist or may not be unique. With teleport, the modified chain is stochastic, irreducible, and aperiodic — by the fundamental theorem of Markov chains, a unique stationary distribution exists and any starting distribution converges to it.

## A11

Algebraic equivalence (no dead-ends): $A = \beta \cdot M + (1-\beta)/N \cdot J$. Then $A \cdot r = \beta \cdot M \cdot r + (1-\beta)/N \cdot J \cdot r$. Since $\Sigma r_i = 1$, $J \cdot r = 1_N$ (the all-ones vector times 1). So $A \cdot r = \beta \cdot M \cdot r + (1-\beta)/N \cdot 1_N$, exactly the tax-and-redistribute form. ✓
With dead-ends, M is sub-stochastic so $M \cdot r$ sums to some $S < 1$. The tax-and-redistribute form would only redistribute $(1-\beta)$ mass when in fact $(1-\beta) + (1-S) \cdot \beta$ mass needs redistributing. The fix is to replace $(1-\beta)/N$ with $(1-S)/N$, where $S = \beta \cdot \Sigma(M \cdot r)$. This guarantees $\Sigma r^{new}=1$.

## A12

**(B)** is correct. (A) is wrong — column-stochastic means columns sum to 1, not rows. (C) is wrong if M has dead-ends — sub-stochastic columns cause leakage. (D) is wrong — column-stochastic only requires non-negative entries, not strictly positive.

## A13

**(A)** the principal eigenvector with eigenvalue 1. (B) is wrong because power iteration specifically picks out the dominant eigenvector. (C) is wrong because if $r^{(0)}$ sums to 1 and M is stochastic, every iterate sums to 1. (D) is wrong because — for an irreducible aperiodic chain — the limit is independent of the starting distribution.

## A14

**False.** Increasing β makes the second-largest eigenvalue of A closer to 1 (the convergence rate is governed by $|\lambda_2| \approx \beta$), so power iteration converges *more slowly*. β=0.85 is a balance: high enough to preserve link signal, low enough to converge in ~50 iterations.

### A15

Per-page leak correction = $(1 - S)/N = (1 - 0.92)/100 = 0.08/100 = 0.0008$. **Answer (A)**. Note: this is independent of $\beta$ — once you have the leaked mass S, you redistribute the missing fraction uniformly across all N pages.

## § 13 ENDING KEY NOTES — REVISION CARDS

Read these in the last 30 minutes before walking into the hall. Every card is one definition / formula / common mistake — bite-sized for last-minute recall.

| | |
|---|---|
| **Out-degree $d_i$**<br>Number of out-links from page i. The denominator in the flow equation. | **Flow equation**<br>$r_j = \Sigma_{i\to j}\, r_i/d_i$. Sum of incoming rank shares. |
| **Stochastic matrix M**<br>$M_{ji} = 1/d_i$ if i→j else 0. Each *column* sums to 1. | **Matrix form**<br>r = M·r. Means r is the eigenvector of M with eigenvalue 1. |
| **Power iteration**<br>$r^{(t+1)} = M\cdot r^{(t)}$. Start uniform, ~50 iters to converge. | **Random walk view**<br>$r_j$ = stationary probability of a random surfer being at page j. |
| **Stationary distribution conditions**<br>Markov chain must be (i) stochastic, (ii) aperiodic, (iii) irreducible. | **Spider trap**<br>Set of pages whose all out-links stay inside. Rank concentrates in the trap. |
| **Dead-end**<br>Page with no out-links. Column of M is all-zero → mass leaks → ranks decay to 0. | **Teleport (Google fix)**<br>With prob. 1−β, jump to a random page. β ≈ 0.8–0.9. Fixes both pathologies. |
| **Google matrix A**<br>$A = \beta\cdot M + (1-\beta)/N \cdot J_{N\times N}$. Stochastic, aperiodic, irreducible. | **Tax & redistribute**<br>$r^{new} = \beta\cdot M\cdot r^{old} + (1-\beta)/N \cdot 1_N$ (no dead-ends). |
| **Leak handling (with dead-ends)**<br>$S = \Sigma(\beta\cdot M\cdot r^{old})$; add (1−S)/N to every entry instead of (1−β)/N. | **Convergence test**<br>$\lvert r^{(t+1)} - r^{(t)}\rvert_1 < \varepsilon$. Common $\varepsilon \approx 10^{-6}$. |
| **Common mistake — out vs in**<br>$r_i/d_i$: $d_i$ is SOURCE i's OUT-degree. Not target's in-degree. | **Common mistake — column vs row**<br>M is column-stochastic. $M_{ji}$ = column i, row j. Many students transpose by mistake. |
| **Eigenvalue intuition**<br>Largest eigenvalue of column-stochastic M is exactly 1. Principal eigenvector = PageRank. | **β semantics**<br>β = expected number of link follows before a teleport. β=0.85 → ~6.7 link clicks on average. |

## § 14 PAGERANK FORMULA & ALGORITHM REFERENCE

Quick-look reference. Memorize the eight rows. Every numerical PageRank exam question reduces to one of them.

| Concept | Formula / Statement | When to use it |
|---|---|---|
| Flow equation | $r_j = \Sigma_{i\to j}\, r_i/d_i$ | Defining PageRank from the link graph. |
| Matrix form | $r = M\cdot r$ where $M_{ji}=1/d_i$ if $i\to j$ else 0 | Compact statement; basis of all algorithms. |
| Eigenvector view | r is the principal eigenvector of M with eigenvalue 1 | Justifying convergence of power iteration. |
| Power iteration | $r^{(t+1)} = M\cdot r^{(t)}$, $r^{(0)}=1/N$ | Practical computation when M is well-behaved. |
| Random walk | r is the stationary distribution of the Markov chain M | Probabilistic interpretation; intuition for teleport. |
| Google formulation | $r = \beta\cdot M\cdot r + (1-\beta)/N \cdot 1_N$ | Default formula when graph has spider traps but no dead-ends. |
| Google matrix | $A = \beta\cdot M + (1-\beta)/N \cdot J_{N\times N}$ | Algebraic / theoretical statement of the teleport fix. |
| Complete algorithm (with leak) | $r^{new}_j = \beta\cdot(M\cdot r^{old})_j + (1-S)/N$ where $S=\Sigma(\beta\cdot M\cdot r^{old})$ | Production form — handles dead-ends without removing them. |
| Convergence threshold | Stop when $\Sigma_j \lvert r^{new}_j - r^{old}_j \rvert < \varepsilon$ | Practical halting criterion. $\varepsilon \approx 10^{-6}$. |

*Table 14.1. PageRank reference card.*

### Algorithmic complexity quick reference:

- One iteration of power method on sparse M: **O(N + E)** arithmetic operations, where E is the number of edges.
- Storage for sparse M: ~10 bytes per edge (with 32-bit indices) → ~100 GB for the 2014 web ($\sim 10^{10}$ edges).
- Typical iterations to converge with $\beta=0.85$: 50–100 to $\varepsilon=10^{-6}$.
- Block-stripe matrix update: split $r^{new}$ into k blocks, multiply M against each in turn — O(N + E) work but only ~N/k working memory.

### Where this connects to other weeks:

- **W12 — Topic-Sensitive PageRank, TrustRank, HITS**: variants of PageRank that bias the teleport vector (instead of uniform 1/N) toward a topic or trust set, plus the complementary Hubs-and-Authorities decomposition. Walk into W12 already owning §3–§9 and you save ~40% of the prep time.
- **W16 — Community Detection**: spectral methods cluster the graph using the same eigenvector machinery introduced here, applied to a different matrix (the graph

Laplacian).

• **MapReduce iteration** (briefly mentioned in slides; depth in MMDS §5.2.2): each iteration's matrix–vector multiply distributes across mappers (one per row of M); reducers sum the contributions. Beyond the slide-level scope but useful for the lab final.